

Software Engineering Fundamentals Project

Group members:

Jana Hassan Qurbi

Maha Safar Almuqati

Maryam Mesfer Aloufi

Layan Faez Al-Swaat

Sham Mastour Al-Harithi

Project Name:

BestPrice

Project idea:

Our project is about creating an app that helps people compare prices before buying a product. When someone wants to buy something, like a phone, they usually check many stores to find the best price. Our app shows prices from different stores such as Jarir, Amazon, and Noon in one place, which makes the process easier and faster. It also analyzes previous prices and discounts to see if the price might decrease soon, and suggests whether it is better to buy now or wait. In addition, the app considers the user's budget and what they need the product for, so it can give suitable recommendations. The main idea is to combine price comparison and smart analysis in one app to support better purchasing decisions.

The problem:

1**. Problems the application solves:**

- Difficulty in comparing prices manually.
- Wasting time searching.
- Fear of buying at an unsuitable price.
- Not knowing the best time to buy.
- Not regularly following promotions and offers.
- Difficulty in choosing the best store.
- Buying without knowing the real need (which means it targets users who purchase products with specifications higher than their actual needs).

2. **Problems the application does not solve:**

- Warranty and maintenance issues later on.
- Errors in store data.

- Differences in return policies.
- Payment or banking issues.
- Differences in service quality within the same store.

Project Scope:

- Develop a smart price comparison and purchase advisor application.
- Allow users to search for electronic products (e.g., smartphones).
- Compare prices across selected local online stores.
- Display historical price trends.
- Provide recommendations (buy now or wait).
- Suggest the best store based on price and offers.
- Limited to electronics and selected local retailers only.

Functional Requirements

1- The user shall be able to create an account:

1.1 The system shall allow the user to enter a phone number, email, and password.
1.2 The system shall verify that the entered email is not already registered in the database.
• If the email already exists, the system shall display the message “This email is already registered.”
• If the email is new, the system shall send a verification code to the entered email.
1.3 The system shall allow the user to enter the verification code.
• If the code is incorrect, the system shall display “Incorrect verification code.”
• If the code is correct, the system shall create the account.
1.4 The system shall display “Account created successfully.”

2- The user shall be able to log in:

2.1 The system shall allow the user to enter email and password.
2.2 The system shall validate the entered credentials.
• If incorrect, the system shall display “Invalid email or password.”
• If correct, the system shall grant access to the user.

3- The user shall be able to search for products:

3.1 The system shall allow the user to enter a product name in the search bar.
3.2 The system shall display a list of matching electronic products.
3.3 The system shall allow filtering results by category or brand.

4- The system shall display product price comparisons:

4.1 The system shall retrieve product prices from multiple stores (e.g., Jarir, Amazon, Noon).
4.2 The system shall display prices in a comparison format.
4.3 The system shall highlight the lowest price among the listed stores.

5- The system shall provide price analysis and predictions:

5.1 The system shall analyze the price history of a selected product over the past months.
5.2 The system shall display a price trend graph (Increase/Decrease).
5.3 The system shall provide a recommendation to the user:
• If the price is at its lowest, the system shall display "Best time to buy now."
• If the price is expected to drop, the system shall display "Better to wait for upcoming discounts."

6- The user shall be able to manage a "Price Alert" (Watchlist):

6.1 The system shall allow the user to add a specific product to their "Watchlist."
6.2 The system shall allow the user to set a "Target Price" for a product.
6.3 The system shall send a notification to the user when the product price reaches the target or drops significantly.
• If the price drops, the system shall display "Price Drop Alert: [Product Name] is now available for [Price]."

7- The system shall provide smart recommendations based on budget:
7.1 The system shall allow the user to enter their maximum budget and the required product category.
7.2 The system shall filter and suggest the best products that fit within the entered budget.
7.3 The system shall display "Top Recommendation" based on the best value-for-money rating.

8- The user shall be able to manage their profile and favorites:
8.1 The system shall allow the user to view and edit their profile information (Name, Email).
8.2 The system shall allow the user to save specific stores as "Favorite Stores."
8.3 The system shall allow the user to log out of the application safely.
• The system shall display a confirmation message: "Are you sure you want to log out?"
• If confirmed, the system shall terminate the session and redirect to the login page.

Non-Functional Requirements

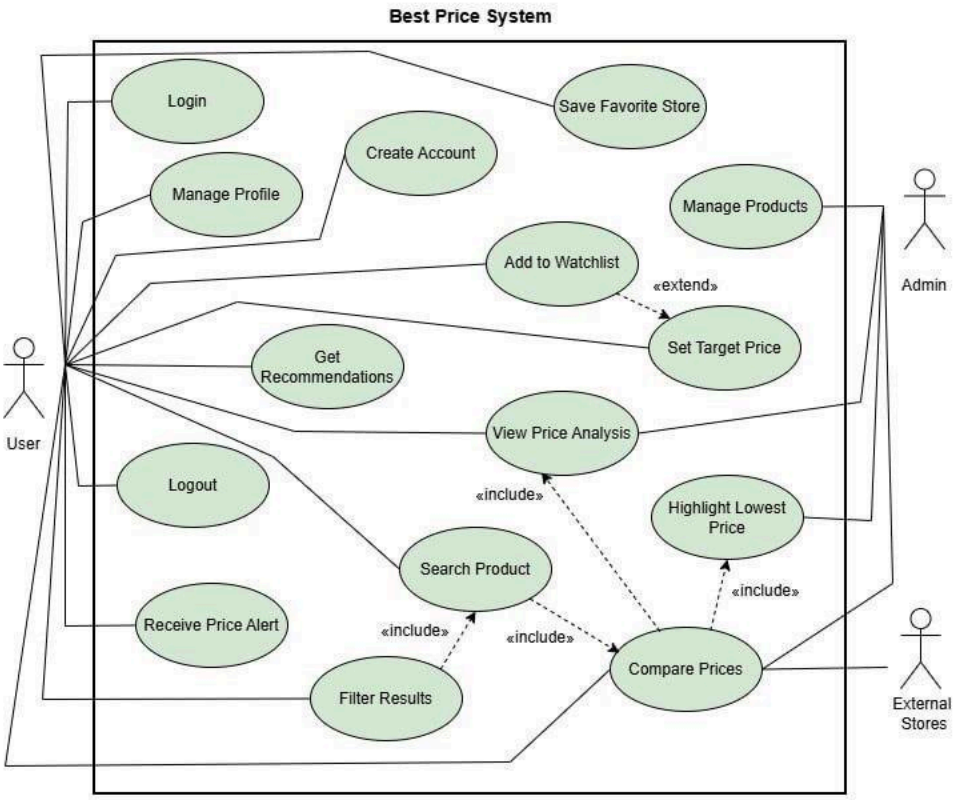
• **Performance:** The system shall return product comparison results within 2 seconds for any search query.
• **Availability:** The system shall maintain at least 99.5% uptime per month, excluding scheduled maintenance.
• **Security:** The system shall require user authentication and encrypted data transmission (HTTPS) before accessing personal data.
• **Usability:** The system shall allow new users to complete a product search within 1 minute without prior training, with a clear and intuitive interface.
• **Reliability:** The system shall ensure no more than one system failure per month and shall automatically recover within 5 minutes.

User Story

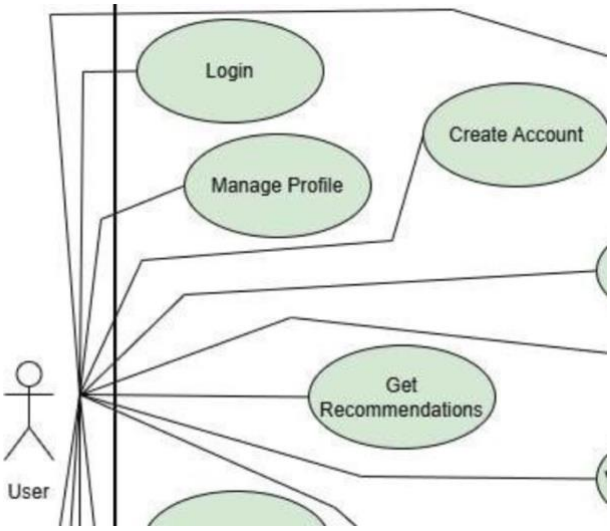
Scenario Name: Compare Product Prices

Item	Description
Starting Situation	User opens the app to find the best price for a product before buying.
Normal Flow	1. The user opens a product page. 2. The system shows prices for the same product from different stores. 3. The user looks at the prices and compares them. 4. The system highlights the lowest price.
What Can Go Wrong	Some prices may not appear, or there could be a connection issue.
Concurrent Activities	While the page is loading, the system is collecting prices from stores and loading product details.
Final State	Success → the user can see the available prices and choose where to buy Failure → The user sees error message

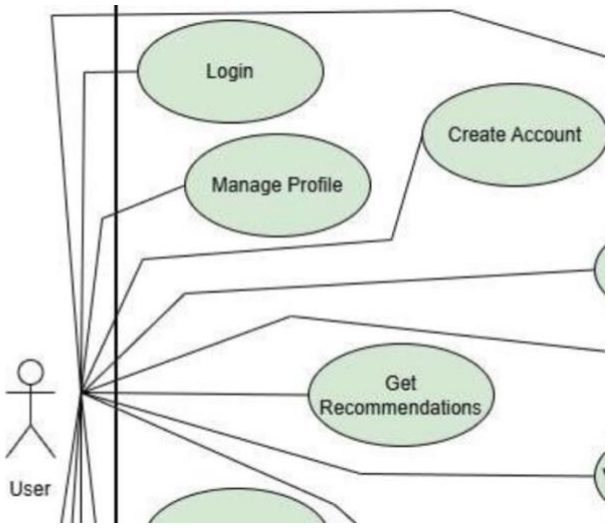
Use Case Diagram



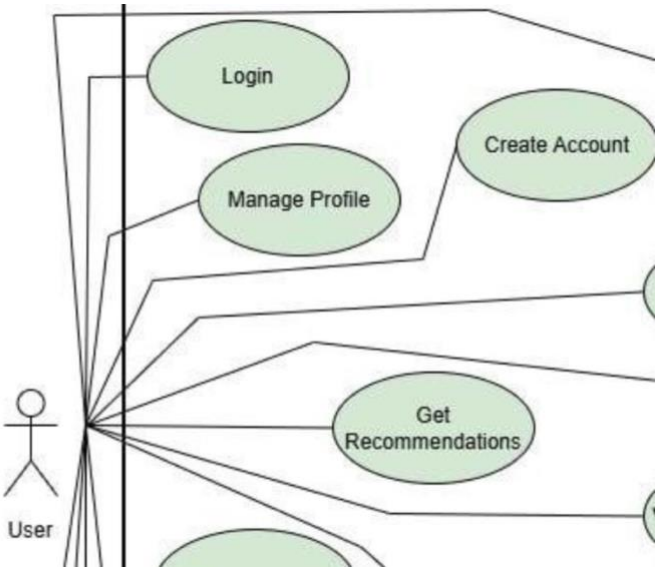
Use Case Description



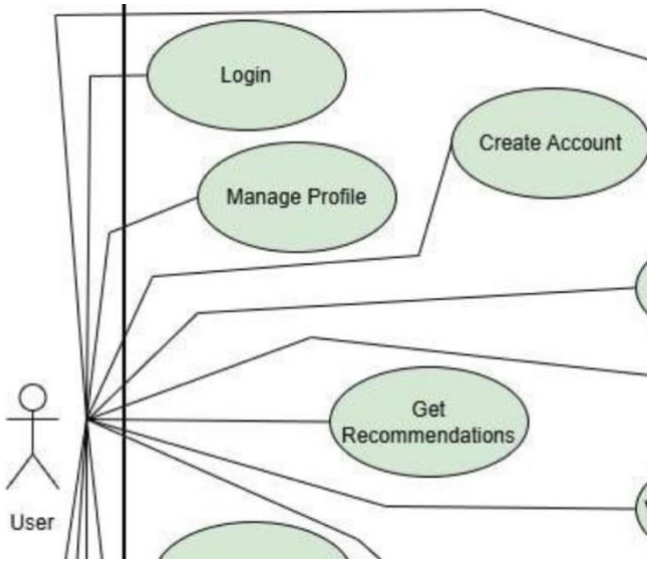
Use Case Name	Actor(s)	Description	Data	Stimulus	Response	Comments
Login	Registered User	User logs into the system	Username, Password	Enter login data	Access granted/denied	Requires account



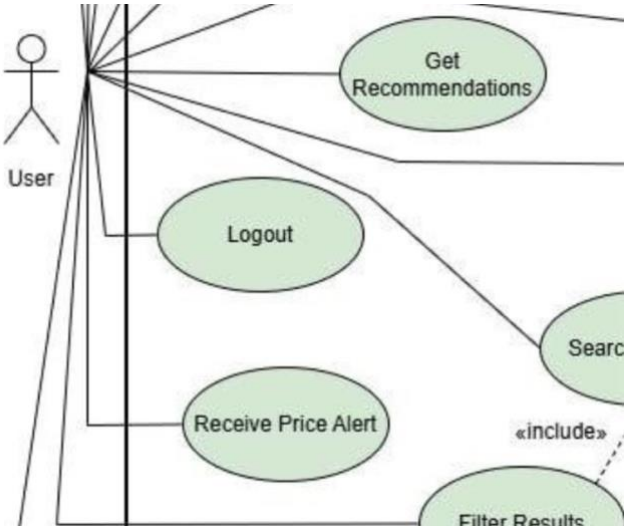
Use Case Name	Actor(s)	Description	Data	Stimulus	Response	Comments
Manage Profile	Registered User	Edit user profile	Profile data	Update info	Saved changes	Requires login



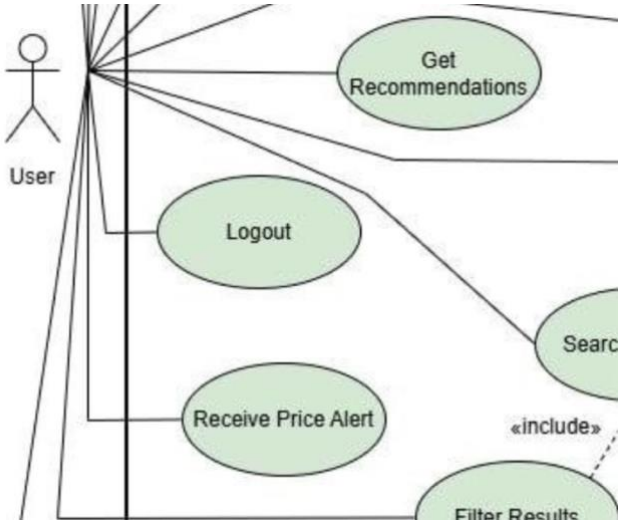
Use Case Name	Actor(s)	Description	Data	Stimulus	Response	Comments
Create Account	New User	Create new account	Name, Email, Password	Choose create account	Account created	Email verification



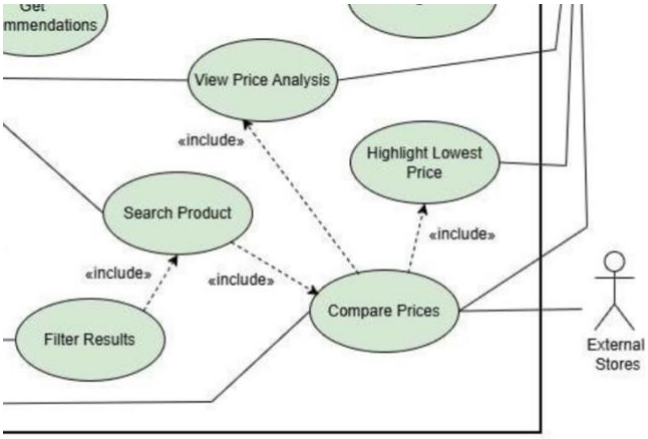
Use Case Name	Actor (s)	Description	Data	Stimulus	Response	Comments
Get Recommendations	User	Get product suggestions	User preferences	Request recommendations	Show suggestions	AI-based



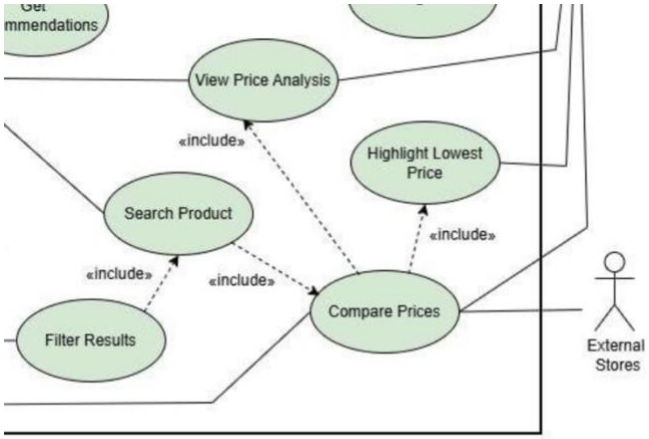
Use Case Name	Actor(s)	Description	Data	Stimulus	Response	Comments
Logout	Registered User	User logs out	Session data	Click logout	Session ended	Security feature



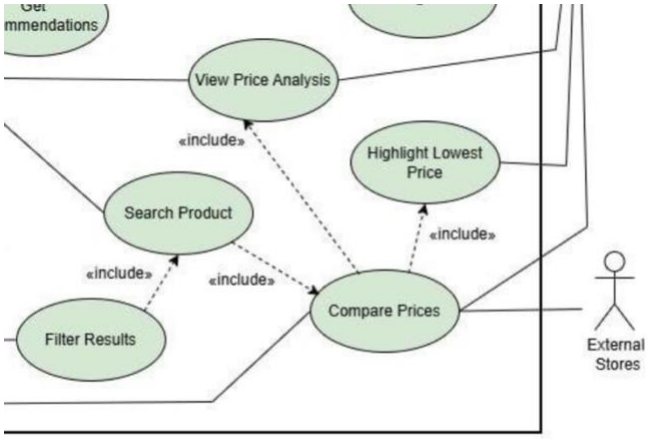
Use Case Name	Actor(s)	Description	Data	Stimulus	Response	Comments
Receive Price Alert	User	Get notified on price drop	Product, target price	Price reaches target	Notification sent	Depends on target price



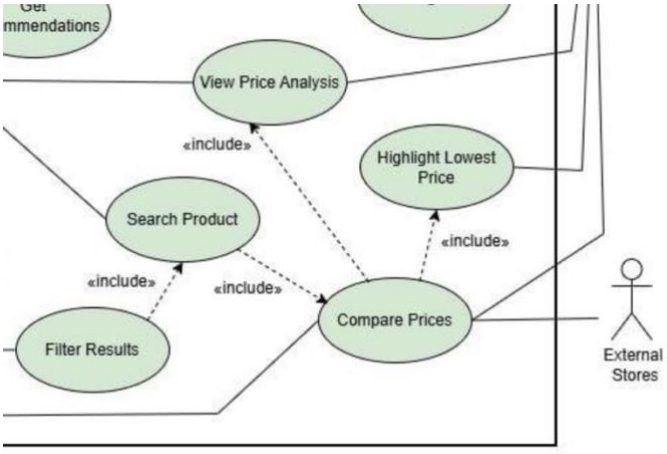
Use Case Name	Actor(s)	Description	Data	Stimulus	Response	Comments
View Price Analysis	User , Admin	View price trends	Historical data	Select analysis	Show analysis	Decision support Include Compare prices



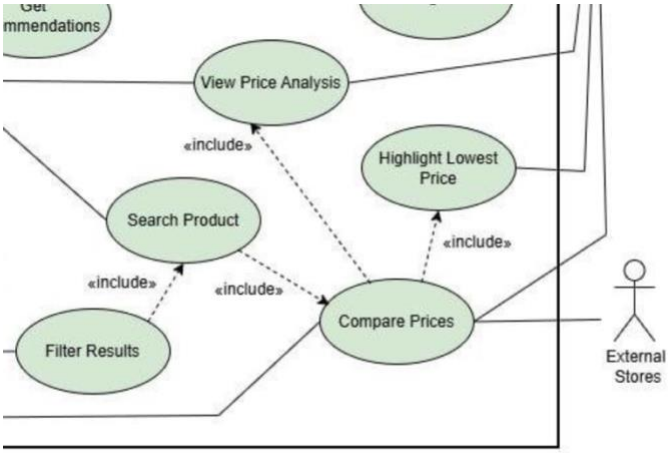
Use Case Name	Actor(s)	Description	Data	Stimulus	Response	Comments
Search Product	User	Search for a product	Keyword	Enter product name	List of products	Can filter results



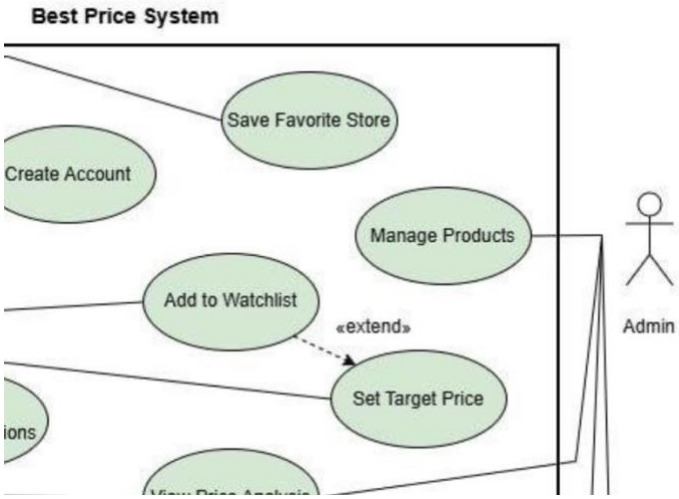
Use Case Name	Actor(s)	Description	Data	Stimulus	Response	Comments
Filter Results	User	Filter search results	Filter criteria	Apply filter	Filtered list	Used with search



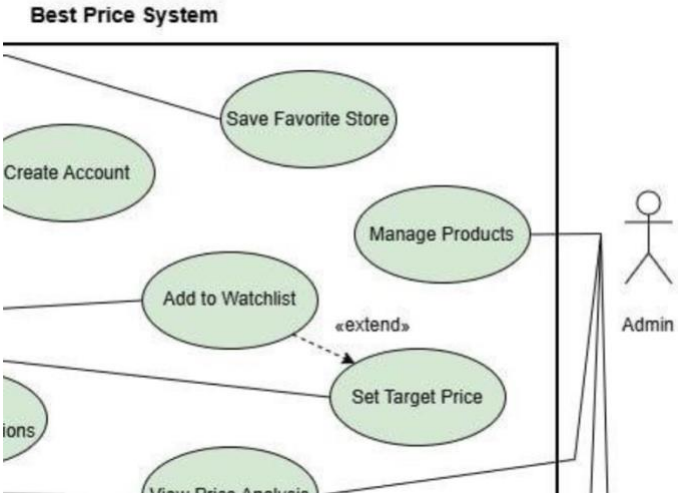
Use Case Name	Actor(s)	Description	Data	Stimulus	Response	Comments
Compare Prices	User, External Stores Admin	Compare product prices	Product info	Select compare	Show prices	Includes Highlights lowest price Include Search product



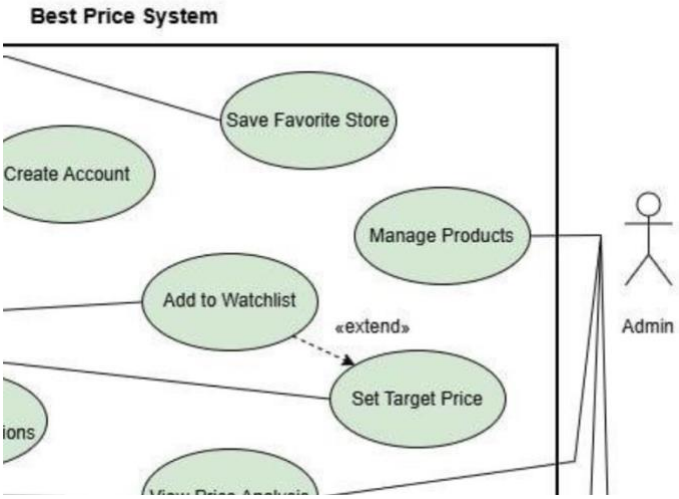
Use Case Name	Actor(s)	Description	Data	Stimulus	Response	Comments
Highlight Lowest Price	Admin	Show cheapest option	Price list	After comparison	Highlight shown	Part of compare price



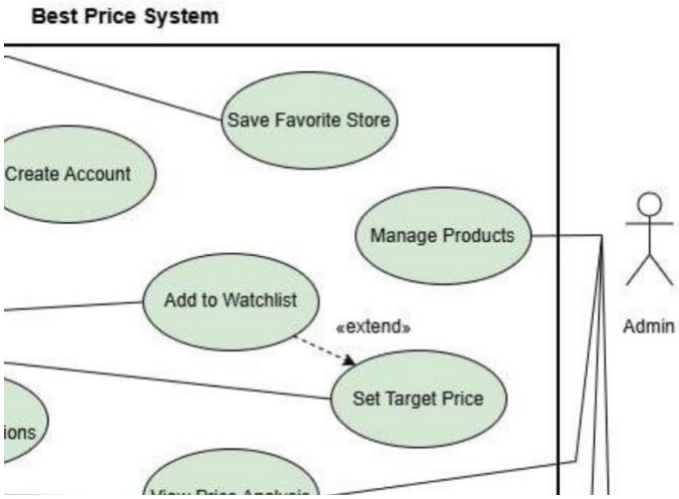
Use Case Name	Actor(s)	Description	Data	Stimulus	Response	Comments
Save Favorite Store	User	Save preferred stores	Store info	Select store	Store saved	Personalization



Use Case Name	Actor(s)	Description	Data	Stimulus	Response	Comments
Add to Watchlist	User	Add product to watchlist	Product ID	Click add	Saved to list	Used for alerts Extend Set Targets price



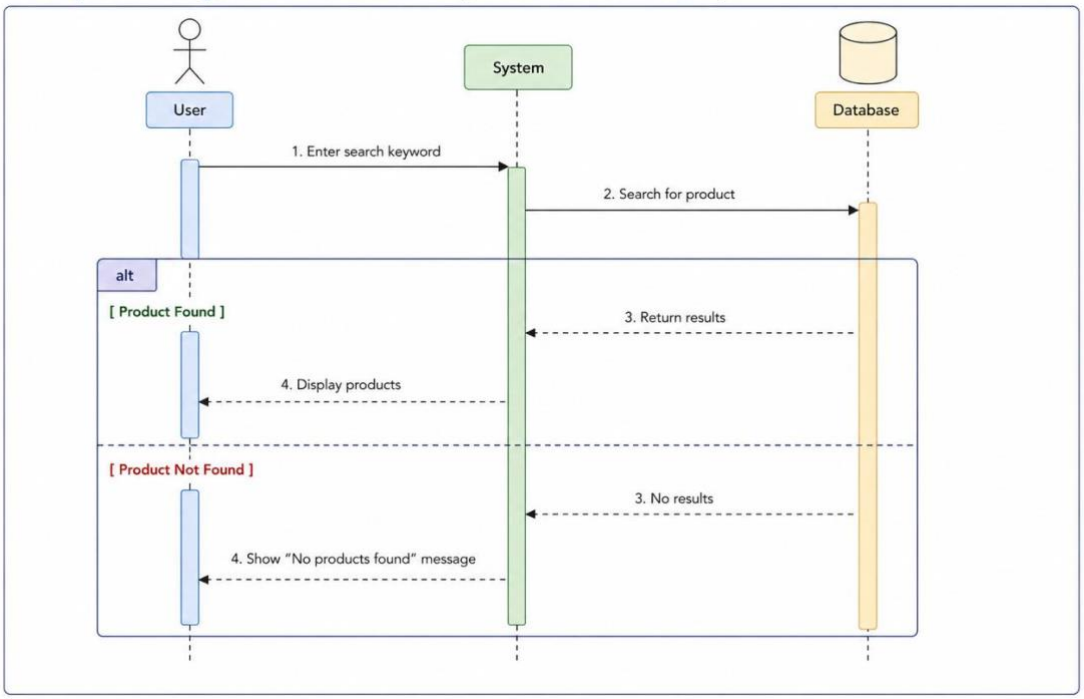
Use Case Name	Actor(s)	Description	Data	Stimulus	Response	Comments
Manage Products	Admin	Admin manages products	Product details	Admin action	Database updated	Admin only



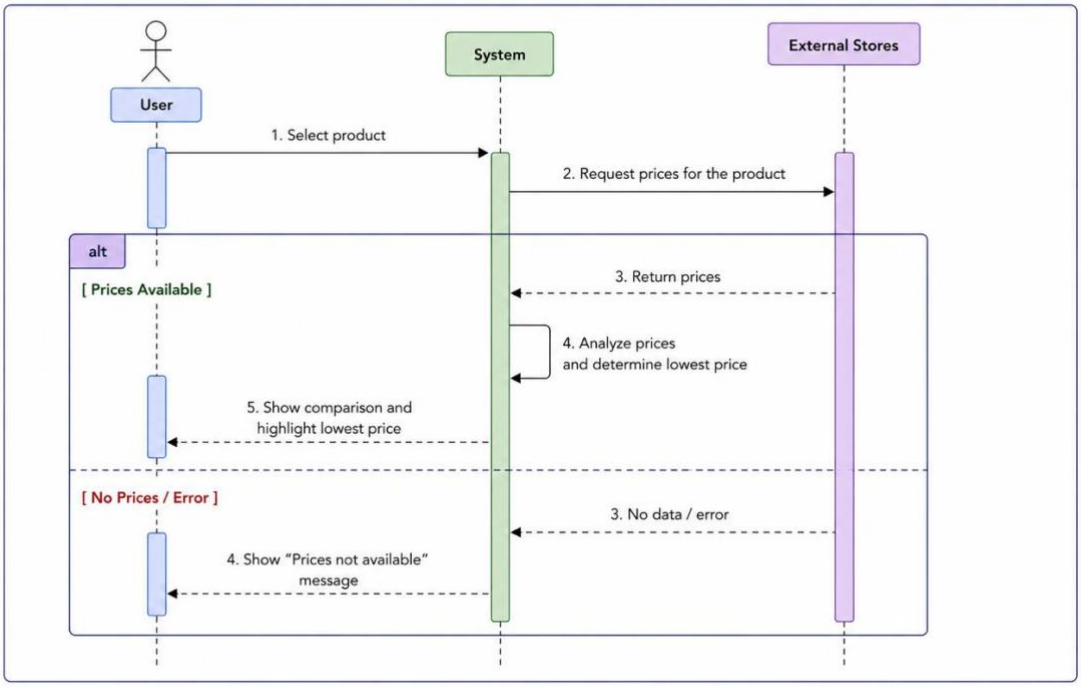
Use Case Name	Actor(s)	Description	Data	Stimulus	Response	Comments
Set Target Price	User	Set desired price	Target price	Enter price	Saved	Linked to alerts

Sequence Diagram

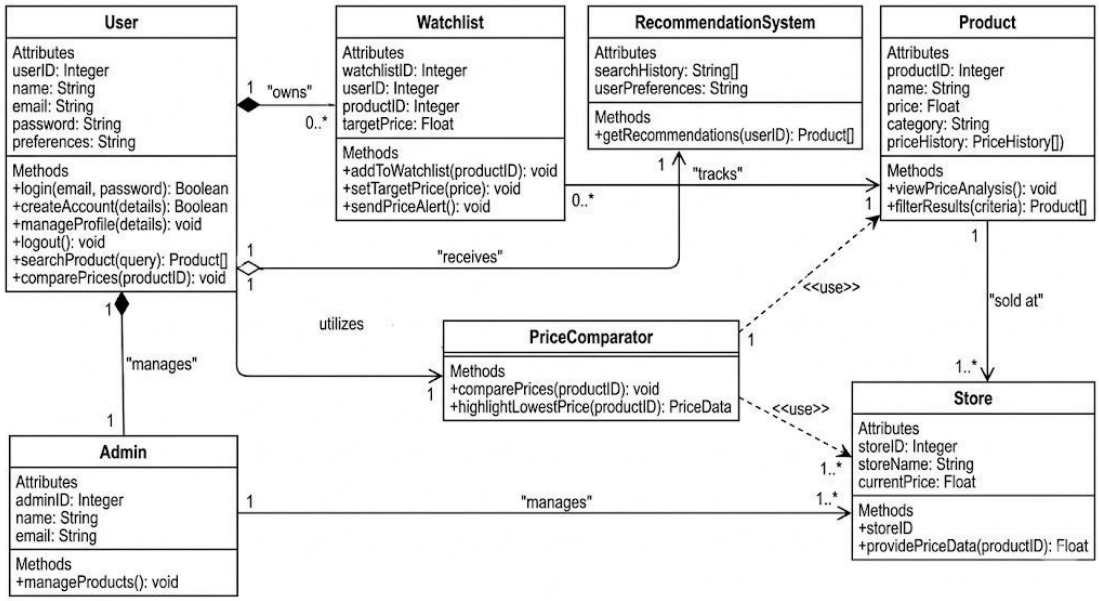
1. Sequence Diagram: Search Product (with Not Found Case)



2. Sequence Diagram: Compare Prices (with Error Case)



Class Diagram

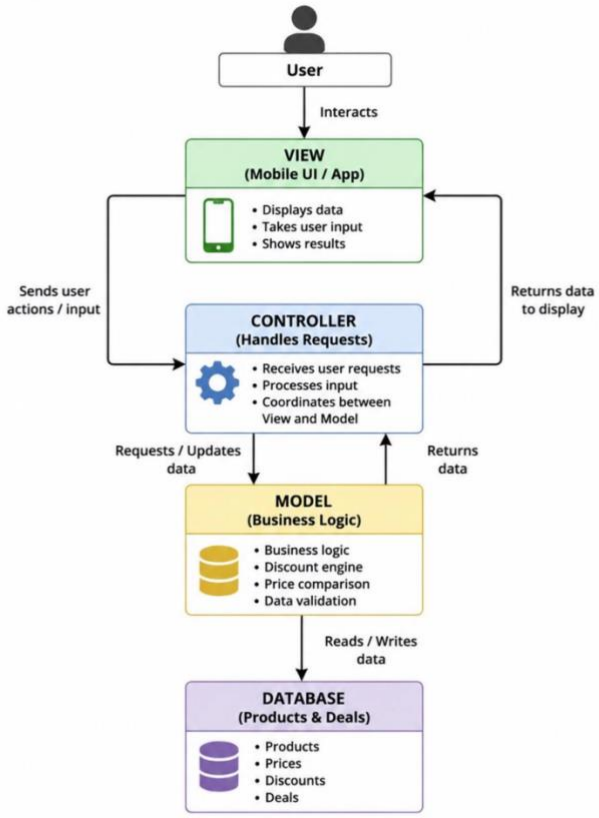


Architectural Pattern

Architectural pattern: **MVC**

We chose MVC for our project because it helps organize the application better. The View handles the user interface, the Controller processes user requests, and the Model manages product and price data. This makes the system easier to manage and update.

System Architecture Diagram



System Components

1. User
The user uses the application to search for products, compare prices, and view discounts from different stores.

2. View (Mobile App Interface)
The View displays products, seasonal offers, discounts, and comparison results to the user through the mobile application interface.

3. Controller
The Controller handles user actions such as searching for products, filtering offers, selecting stores, and requesting price comparisons.

4. Model (Discount Logic & Comparison)
The Model processes product and discount data, compares prices between stores, and identifies the best available offer.

5. Database (Stores & Offers Data)
The Database stores information about products, stores, prices, discounts, and seasonal offers used by the system.